

# Markov normal algorithms

## Exercise sheet 4

Fix a finite working alphabet  $\Gamma$ . A **Markov normal algorithm** is an ordered finite list of rules

$$u \rightarrow v \quad \text{or} \quad u \rightarrow \cdot v, \quad u, v \in \Gamma^*.$$

The second form is called a *terminal rule*. To make one step on a word  $w$ , take the **first** rule in the list whose left-hand side occurs in  $w$ , and replace its **leftmost** occurrence. Then start scanning the rule list again from the top. The computation stops when no rule applies, or immediately after a terminal rule is used; the current word is the output. The empty word  $\varepsilon$  is allowed on either side of a rule, and an empty left-hand side matches at the beginning of the word. The input alphabet may be a subset of  $\Gamma$ , so auxiliary symbols are allowed.

**Notation:**  $a^n$  is a word of  $n$  consecutive  $a$ 's:  $\underbrace{aa \dots a}$ .

**Exercise 4.1.**<sup>1</sup> Given a word  $w \in \{a, b, c\}^*$ , rearrange its letters into nondecreasing order. For example,  $cbacba \mapsto aabbcc$ .

**Exercise 4.2.** The input is  $o^n$ . Output the word **odd** when  $n$  is odd and **even** when  $n$  is even.

**Exercise 4.3.** The input is the standard binary representation of an integer  $n$ . Output the binary representation of  $n + 1$ .

**Exercise 4.4.** The input is  $o^n$ . Output the usual decimal representation of  $n$ .

**Exercise 4.5.** Duplicate a binary word  $w$ :  $w \mapsto ww$ .

**Exercise 4.6.** Given a binary word  $w$ , output **yes** if  $w$  is a palindrome, and **no** otherwise.

**Exercise 4.7.** Reverse a binary word  $w$ : e.g.  $111001 \mapsto 100111$ .

**Exercise 4.8.** The input has the form  $o^n \# o^m$ . Output  $o^{\gcd(n,m)}$ .

**Exercise 4.9.** Construct a normal Markov algorithm that evaluates a Boolean expression in prefix Polish notation over  $\{0, 1, \wedge, \vee\}$ , leaving its value, 0 or 1, as the result.

**Exercise 4.10.** \* The input is  $o^{n^2}$ . Output  $o^n$ .

**Exercise 4.11.** \* Prove that normal algorithms are Turing-complete. That is, every computable function is computed by some normal algorithm.

---

<sup>1</sup>Exercises 1-10 are adapted from <https://mao.snuke.org/tasks/>.